



**Tecnológico
de Monterrey**

**Programación de estructuras de datos y
algoritmos fundamentales**

Gpo 850

Después de clase | Act-Integradora-4 Grafos y
sus algoritmos

Reflexión

Alumno: Carlos Alberto Pérez Díaz

Matrícula: A01204060

Fecha: 26/05/2026

A. Importancia y eficiencia del uso de grafos en esta problemática

El uso de grafos en esta actividad resulta fundamental porque permite modelar de forma natural las conexiones entre direcciones IP dentro de una red. En este caso, cada dirección IP representa un nodo y cada conexión o incidencia registrada en la bitácora representa una arista dirigida con un peso asociado. Esta representación facilita el análisis de relaciones entre equipos, tráfico de red y posibles patrones sospechosos.

Además, los grafos permiten trabajar eficientemente con grandes cantidades de conexiones. Gracias a la representación mediante listas de adyacencia, el consumo de memoria es mucho menor que utilizando matrices de adyacencia, especialmente cuando el número de conexiones reales es significativamente menor que el número máximo posible de conexiones.

Otra ventaja importante es que los grafos permiten aplicar algoritmos especializados como Dijkstra para encontrar caminos mínimos entre nodos. Para esta actividad, esto ayuda a determinar qué direcciones IP son más difíciles de alcanzar desde el bot master y analizar posibles rutas de propagación dentro de la red. De esta manera, los grafos no solo almacenan información, sino que también permiten obtener conclusiones relevantes sobre el comportamiento de la red y posibles amenazas de seguridad.

B. Reflexión sobre el uso de Binary Heap

Para resolver esta actividad se decidí utilizar un Binary Heap debido a que permite obtener eficientemente los nodos con mayor grado de salida. El heap es especialmente adecuado cuando se necesita recuperar repetidamente el elemento con mayor prioridad, en este caso la IP con mayor cantidad de conexiones salientes.

Las operaciones principales de un Binary Heap tienen complejidad:

- $\text{push}() \rightarrow O(\log n)$
- $\text{pop}() \rightarrow O(\log n)$
- $\text{getTop}() \rightarrow O(1)$

Comparado con otras estructuras jerárquicas, como un BST, el Binary Heap resulta más eficiente para esta necesidad específica. Aunque un BST balanceado también puede realizar inserciones y búsquedas en $O(\log n)$, obtener continuamente el valor máximo puede implicar recorridos adicionales dependiendo de la implementación.

Otra ventaja importante es que el Binary Heap utiliza almacenamiento compacto mediante un vector, reduciendo el uso de memoria en comparación con estructuras basadas en nodos dinámicos. Esto resulta útil considerando el tamaño de la bitácora utilizada en la actividad.

Por estas razones, el uso del Binary Heap fue la decisión correcta para identificar rápidamente las IPs con mayor grado de salida y determinar el posible bot master.

C. Complejidad computacional de las operaciones básicas

La implementación desarrollada utiliza principalmente dos estructuras de datos: el grafo mediante listas de adyacencia y el Binary Heap.

Carga de la bitácora:

La carga del archivo y construcción del grafo tiene complejidad aproximada $O(m)$, donde m representa el número de incidencias registradas en la bitácora.

Cálculo de grados:

El cálculo de grados de salida tiene complejidad $O(n)$.

Dijkstra:

El algoritmo de Dijkstra implementado con una priority queue tiene complejidad $O((n + m) \log n)$.

Operaciones del Binary Heap:

- $\text{push}() \rightarrow O(\log n)$
- $\text{pop}() \rightarrow O(\log n)$
- $\text{getTop}() \rightarrow O(1)$

En conjunto, el uso de listas de adyacencia y Binary Heap permitió desarrollar una solución eficiente tanto en tiempo de ejecución como en consumo de memoria.

D. Reflexión sobre el algoritmo para caminos mínimos

Para encontrar el camino más corto entre el bot master y el resto de las IPs Utilice el algoritmo de Dijkstra. Este seleccione este algoritmo porque el grafo de la actividad utiliza pesos no negativos en las conexiones, condición necesaria para el correcto funcionamiento de Dijkstra.

La principal ventaja de este algoritmo es que garantiza encontrar las distancias mínimas desde un nodo origen hacia todos los demás nodos del grafo.

La implementación usando una priority queue mejora considerablemente el rendimiento del algoritmo, alcanzando una complejidad $O((n + m) \log n)$.

Comparado con otros algoritmos:

- BFS no es adecuado porque solo funciona correctamente en grafos sin pesos.
- Floyd Warshall tiene complejidad $O(n^3)$.
- Bellman Ford tiene complejidad $O(nm)$.

Por estas razones, Dijkstra representó la alternativa más eficiente.

E. Pseudocódigo y complejidad temporal

Algoritmo guardarCaminoAtaque

Entrada:

ipDestino

1. Obtener el índice correspondiente a la IP destino.
2. Crear un vector llamado camino.
3. Mientras el nodo actual sea diferente de -1:
 agregar nodo actual al vector camino
 actual = predecesores[actual]
4. Invertir el vector camino.
5. Guardar cada IP del camino en el archivo ataque_botmaster.txt.
6. Fin.

La complejidad temporal de este procedimiento es $O(n)$, ya que en el peor caso el camino reconstruido podría incluir todos los nodos del grafo.

Referencias bibliográficas

- Cormen, T., Leiserson, C., Rivest, R., & Stein, C. Introduction to Algorithms. MIT Press.
- Weiss, M. Data Structures and Algorithm Analysis in C++. Pearson.
- cppreference.com
- Material y códigos vistos en clase del curso.
- ChatGPT, apoyo para organización, documentación y revisión del código.

<https://chatgpt.com/share/6a1656ec-dfe8-83e8-ac4a-226ef215e5eb>